Soft Actor-Critic (SAC): Complete Derivation and Design Motivations

Course home · 简体中文 · PDF

Item	Details
Applicability	Modern continuous-action SAC; twin critics and automatic temperature tuning; pseudocode only
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Starting from Standard Reinforcement Learning
- 2. Step One: Maximum-Entropy Reinforcement Learning
  - 2.1 Policy entropy and the temperature parameter
  - 2.2 Rewriting entropy as a sample-based expectation
- 3. Step Two: Deriving the Soft Bellman Equation
  - 3.1 Comparison with ordinary Bellman equations
  - 3.2 Defining soft V and soft Q consistently
  - 3.3 Soft Bellman expectation equation
- 4. Step Three: Soft Policy Improvement
  - 4.1 Why ordinary greedy improvement is insufficient
  - 4.2 Calculus of variations yields the Boltzmann policy
- 5. Step Four: Turning Policy Improvement into KL Minimization
  - 5.1 Why directly computing the Boltzmann policy is difficult
  - 5.2 Why exact soft policy iteration improves value
- 6. Step Five: From Soft Policy Iteration to Actor-Critic
- 7. Step Six: Why SAC Can Learn Off-Policy
  - 7.1 PPO and policy-dependent data
  - 7.2 Why an old transition can support a new Bellman update
- 8. Step Seven: Deriving the Complete Critic Loss
  - 8.1 From the soft Bellman target to TD regression
  - 8.2 Target networks reduce target drift

    - 8.3 Clipped double Q mitigates overestimation
    - 8.4 Final critic target and losses
- 9. Step Eight: Deriving the Actor Loss
- 10. Step Nine: The Reparameterization Trick
    - 10.1 Turning parameter-dependent sampling into a differentiable function
    - 10.2 A tanh-squashed Gaussian policy
    - 10.3 Rewriting the actor loss as a pathwise objective
- 11. Step Ten: Correcting the Tanh-Gaussian Log Probability
- 12. Step Eleven: Automatic Entropy Tuning
    - 12.1 From fixed temperature to an entropy constraint
    - 12.2 The temperature loss and its direction of change
    - 12.3 Why optimize log-alpha?
    - 12.4 Choosing target entropy
- 13. Step Twelve: Putting Together the Complete Modern SAC Algorithm
    - 13.1 Components and three optimization problems
    - 13.2 Complete sequence for one update
    - 13.3 Instructional pseudocode
- 14. Original SAC v1 Versus Modern SAC
    - 14.1 Original SAC v1: explicitly learning a V-network
    - 14.2 Modern SAC: remove the explicit V-network
- 15. Fundamental Differences Between SAC and PPO
    - 15.1 Comparing policy-update equations
    - 15.2 Comparison table
    - 15.3 Why PPO remains popular in robot simulation
- 16. Final Summary: Five Essential Formula Groups
    - The single most important PPO/SAC distinction
- References

1. Starting from Standard Reinforcement Learning

Soft Actor-Critic (SAC) is an **off-policy, stochastic-policy, actor-critic** reinforcement-learning algorithm developed primarily for continuous control. It maximizes long-term reward while encouraging an appropriate amount of policy randomness. Soft Bellman backups provide action-value estimates, and the actor is optimized using gradients through a differentiable Q-function. The full method combines maximum-entropy objectives, soft policy iteration, a replay buffer, two Q-networks, target networks, the reparameterization trick, and automatic entropy-temperature adjustment.

Consider a discounted Markov decision process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

Here $s_t \in \mathcal{S}$ is a state, $a_t \in \mathcal{A}$ is an action, $P(s_{t+1} \mid s_t, a_t)$ is the environment transition kernel, $r(s_t, a_t)$ is the immediate reward, and $\pi_\theta(a \mid s)$ is a policy parameterized by θ. Conventional RL maximizes the discounted return:

$$\boxed{J_{\mathrm{RL}}(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right].}$$

Motivation: why might maximizing reward alone be insufficient? Consider a robot choosing among three grasping actions, with the following current critic estimates:

Candidate action	Estimated Q-value
Grasp from the left	10
Grasp from the right	9.8
Grasp from above	9.5

A policy that immediately commits to the left grasp may stop exploring alternatives. Worse, the critic could be incorrect. For example, it may predict $\hat{Q}(s, a_{\text{left}}) = 10$ and $\hat{Q}(s, a_{\text{right}}) = 8$, even though their true values are $Q(s, a_{\text{left}}) = 8$ and $Q(s, a_{\text{right}}) = 10$. Rarely trying the right grasp makes it harder to correct this error.

We therefore want the policy to prefer high-value actions without eliminating randomness too early. This is the central motivation for **maximum-entropy reinforcement learning**.

2. Step One: Maximum-Entropy Reinforcement Learning

2.1 Policy entropy and the temperature parameter

For discrete actions, Shannon entropy is $\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$. For continuous actions, SAC uses differential entropy:

$$\boxed{\mathcal{H}(\pi(\cdot \mid s)) = -\int \pi(a \mid s) \log \pi(a \mid s)\, da = \mathbb{E}_{a \sim \pi}[-\log \pi(a \mid s)].}$$

For a one-dimensional Gaussian policy $\pi(a \mid s) = \mathcal{N}(\mu(s), \sigma^2(s))$:

$$\mathcal{H}(\pi) = \frac{1}{2}\log(2\pi e \sigma^2) = \log \sigma + \frac{1}{2}\log(2\pi e).$$

A larger standard deviation yields a larger entropy and a more dispersed action distribution. **Importantly**, differential entropy can be negative and changes under rescaling of the action coordinates; it is not identical to discrete entropy.

Introduce the temperature $\alpha \geq 0$ and modify the objective:

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\big)\right].}$$

The coefficient α controls the trade-off between reward and randomness. The ordinary RL objective is approached as $\alpha \to 0$. Larger values encourage more stochastic policies. The coefficient is also important because reward magnitude and entropy magnitude can differ substantially. For example, if $r \approx 100$ while $\mathcal{H}(\pi) \approx 5$, adding the two unweighted may give entropy little influence; α adjusts the relative scale.

2.2 Rewriting entropy as a sample-based expectation

Because $\mathcal{H}(\pi(\cdot \mid s)) = \mathbb{E}_{a\sim\pi}[-\log \pi(a \mid s)]$, we can write:

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) - \alpha \log \pi(a_t \mid s_t)\big)\right].} \tag{1}$$

One may interpret $\tilde{r}_t = r_t - \alpha \log \pi(a_t \mid s_t)$ as an entropy-regularized reward expression. However, this term depends on the current policy and is **not** an external environment reward fixed independently of the policy. The useful change is that continuous-control algorithms now only need to sample $a \sim \pi$ and evaluate $\log \pi(a \mid s)$, rather than explicitly integrating entropy over the entire action space.

3. Step Two: Deriving the Soft Bellman Equation

3.1 Comparison with ordinary Bellman equations

Standard value functions satisfy:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)], \quad Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

Combining them gives:

$$Q^\pi(s,a) = \mathbb{E}_{s',a'}[r(s,a) + \gamma Q^\pi(s',a')].$$

For maximum-entropy RL, future policy entropy must also appear in the recursive value calculation.

3.2 Defining soft V and soft Q consistently

A convenient SAC convention is:

$$\boxed{V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big) \,\middle|\, s_0 = s\right],}$$

$$\boxed{Q^\pi(s,a) = \mathbb{E}_\pi\left[r_0 + \sum_{t=1}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big) \,\middle|\, s_0 = s, a_0 = a\right].}$$

The soft state value includes the entropy term for the current action. The soft action value conditions on a fixed initial action, so its entropy terms start at the next step. This convention makes the recursion concise. By conditioning on the first action:

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].} \tag{2}$$

Equivalently:

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)] + \alpha\mathcal{H}(\pi(\cdot \mid s)).} \tag{3}$$

An ordinary state value measures expected future reward under a policy. A soft state value measures expected reward **together with** the randomness retained by that policy.

3.3 Soft Bellman expectation equation

The definitions and the Markov property imply:

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

Substituting Equation (2):

$$\boxed{Q^\pi(s,a) = \mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}}[r(s,a) + \gamma\left(Q^\pi(s',a') - \alpha\log\pi(a' \mid s')\right)].} \tag{4}$$

Compared with ordinary Bellman evaluation, we have one additional term: $-\alpha\log\pi(a' \mid s')$. It is precisely how the entropy reward enters the critic's target through the next state's soft value. This equation provides **policy evaluation**, but we must still derive how to improve the policy.

4. Step Three: Soft Policy Improvement

4.1 Why ordinary greedy improvement is insufficient

Ordinary policy iteration can choose $\pi_{\text{new}}(s) \in \arg\max_a Q^{\pi_{\text{old}}}(s,a)$. Even when optimizing a stochastic distribution, maximizing $\mathbb{E}_a Q(s,a)$ alone may collapse the solution into a deterministic distribution concentrated on Q-maximizing actions. SAC wants to improve value while retaining entropy.

Fix a state s and define $Q_{\text{old}}(s,a) = Q^{\pi_{\text{old}}}(s,a)$. The soft policy-improvement problem is:

$$\boxed{\pi_{\text{new}} = \arg\max_\pi \left\{\mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)] + \alpha\mathcal{H}(\pi(\cdot \mid s))\right\}.} \tag{5}$$

Equivalently, maximize the functional:

$$F(\pi) = \int \pi(a \mid s)Q_{\text{old}}(s,a)\,da - \alpha\int \pi(a \mid s)\log\pi(a \mid s)\,da$$

subject to the density-normalization constraint $\int \pi(a \mid s)\,da = 1$.

4.2 Calculus of variations yields the Boltzmann policy

Assume $\alpha > 0$ and a finite normalizing constant. Introduce a Lagrange multiplier λ:

$$\begin{aligned}\mathcal{L}(\pi,\lambda) = &\int \pi(a \mid s)Q_{\text{old}}(s,a)\,da \\ &- \alpha\int \pi(a \mid s)\log\pi(a \mid s)\,da \\ &+ \lambda\left(\int \pi(a \mid s)\,da - 1\right).\end{aligned}$$

The functional derivative with respect to the density is:

$$\frac{\delta \mathcal{L}}{\delta \pi(a \mid s)} = Q_{\text{old}}(s,a) - \alpha(\log \pi(a \mid s) + 1) + \lambda.$$

Setting it to zero:

$$\log \pi(a \mid s) = \frac{Q_{\text{old}}(s,a)}{\alpha} + \frac{\lambda}{\alpha} - 1.$$

Exponentiating and normalizing gives:

$$\boxed{\pi^*_{\text{new}}(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z_{\text{old}}(s)}, \quad Z_{\text{old}}(s) = \int_{\mathcal{A}} \exp(Q_{\text{old}}(s,a')/\alpha)\, da'.} \tag{6}$$

This is a **Boltzmann policy**. Higher Q-value means higher action probability. Increasing α produces a flatter distribution; as $\alpha \to 0^+$, probability concentrates on Q-maximizing actions. For example, if $Q(a_1) = 2$ and $Q(a_2) = 1$, then for $\alpha = 0.5$ the probabilities are approximately 0.881 and 0.119. With $\alpha = 2$, they become approximately 0.622 and 0.378.

Motivation recap: soft policy improvement no longer merely chooses $\arg\max_a Q(s,a)$. It learns a distribution that prioritizes high-value actions while preserving randomness.

5. Step Four: Turning Policy Improvement into KL Minimization

5.1 Why directly computing the Boltzmann policy is difficult

For a high-dimensional continuous action space, $Z(s) = \int e^{Q(s,a)/\alpha}\, da$ is generally intractable. We therefore cannot easily construct or sample the normalized ideal distribution. SAC solves this by projecting a parameterized actor π_θ onto the Boltzmann distribution in a way that eliminates the need to calculate $Z(s)$.

Define:

$$\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z(s)}.$$

Consider the KL divergence with the actor as the first argument, $D_{\text{KL}}(\pi_\theta \| \pi_B)$:

$$\begin{aligned} D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a \sim \pi_\theta}\left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s)\right] \\ &= \mathbb{E}_{a \sim \pi_\theta}\left[\log \pi_\theta(a \mid s) - \frac{Q_{\text{old}}(s,a)}{\alpha}\right] + \log Z(s). \end{aligned}$$

During the projection Q_{old} is fixed and $\log Z(s)$ does not depend on the actor parameters. Therefore:

$$\boxed{\arg\min_\theta D_{\text{KL}}(\pi_\theta \| \pi_B) = \arg\min_\theta \mathbb{E}_{a \sim \pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)\right].} \tag{7}$$

This is the origin of the **SAC actor loss**: we neither compute a partition function nor enumerate all high-dimensional actions.

A useful identity makes the equivalence explicit:

$$\boxed{\mathbb{E}_{a \sim \pi_\theta}[Q(s,a)] + \alpha \mathcal{H}(\pi_\theta(\cdot \mid s)) = \alpha \log Z(s) - \alpha D_{\text{KL}}(\pi_\theta \| \pi_B).} \tag{8}$$

Thus, maximizing soft value and approaching the Boltzmann policy are two views of the same optimization problem.

5.2 Why exact soft policy iteration improves value

Assume the old policy belongs to the allowed policy class and that the KL projection is solved exactly at every state. By optimality:

$$D_{\mathrm{KL}}(\pi_{\mathrm{new}} \| \pi_B) \leq D_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_B).$$

Equation (8) then implies:

$$\boxed{\mathbb{E}_{a \sim \pi_{\mathrm{new}}}[Q^{\pi_{\mathrm{old}}}(s,a) - \alpha \log \pi_{\mathrm{new}}(a \mid s)] \geq V^{\pi_{\mathrm{old}}}(s).} \tag{9}$$

Define the soft Bellman operator for a fixed policy π:

$$(\mathcal{T}^{\pi} Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s', a' \sim \pi}\left[Q(s',a') - \alpha \log \pi(a' \mid s')\right].$$

Combining Equation (9) with Bellman evaluation gives:

$$Q^{\pi_{\mathrm{old}}} \leq \mathcal{T}^{\pi_{\mathrm{new}}} Q^{\pi_{\mathrm{old}}}.$$

The operator is monotone. Under suitable boundedness assumptions, it is also a contraction:

$$\|\mathcal{T}^{\pi} Q_1 - \mathcal{T}^{\pi} Q_2\|_{\infty} \leq \gamma \|Q_1 - Q_2\|_{\infty}.$$

Repeated application of $\mathcal{T}^{\pi_{\mathrm{new}}}$ gives:

$$Q^{\pi_{\mathrm{old}}} \leq \mathcal{T}^{\pi_{\mathrm{new}}} Q^{\pi_{\mathrm{old}}} \leq \cdots \longrightarrow Q^{\pi_{\mathrm{new}}}.$$

Consequently:

$$\boxed{Q^{\pi_{\mathrm{new}}}(s,a) \geq Q^{\pi_{\mathrm{old}}}(s,a).} \tag{10}$$

This is the essential soft policy-improvement theorem. **Scope of the result:** the statement assumes exact evaluation and exact improvement under the stated conditions. Finite data, neural function approximation, stochastic gradients, and approximate projection in practical SAC do not inherit the same global convergence guarantee.

6. Step Five: From Soft Policy Iteration to Actor-Critic

We now have two alternating operations:

- **Policy evaluation:** evaluate $Q^{\pi}(s,a) = \mathbb{E}[r + \gamma(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s'))]$ for a fixed policy.
- **Policy improvement:** improve the policy by minimizing $\mathbb{E}_{a \sim \pi}[\alpha \log \pi(a \mid s) - Q(s,a)]$.

A tabular representation is infeasible for general continuous states and actions, so SAC uses neural-network approximations:

$$\boxed{\text{Actor: } \pi_{\theta}(a \mid s), \qquad \text{Critic: } Q_{\phi}(s,a).}$$

The actor takes a state and outputs an action distribution; the critic takes a state-action pair and outputs a scalar value. The soft Bellman equation suggests a temporal-difference error:

$$\delta = Q_\phi(s,a) - \left[r + \gamma\left(Q_\phi(s',a') - \alpha \log \pi_\theta(a' \mid s')\right)\right].$$

One can train the critic with $\frac{1}{2}\mathbb{E}[\delta^2]$ and train the actor with $\mathbb{E}[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)]$. The environment's reward affects the actor indirectly through $r \to Q_\phi \to \pi_\theta$; the environment reward is not directly backpropagated into the policy.

A practical issue remains: if a constantly changing Q_ϕ is used both as the predictor and as its own regression target, learning can be unstable. Section 8 introduces target networks and double-Q learning.

7. Step Six: Why SAC Can Learn Off-Policy

7.1 PPO and policy-dependent data

PPO's clipped objective depends on data collected by the old policy, and on the importance ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

As a result, standard PPO predominantly uses recent on-policy rollouts. Very old data can cause substantial distribution mismatch.

7.2 Why an old transition can support a new Bellman update

In a fixed MDP, the environment transition law $P(s' \mid s,a)$ does **not** depend on which behavior policy generated the state-action pair. A previously observed transition (s,a,r,s') therefore remains a valid sample of the environment dynamics. To construct the **current policy's** soft Bellman target, we simply resample a *new next action* $a' \sim \pi_\theta(\cdot \mid s')$:

$$y = r + \gamma\big(Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\big).$$

Variable	Source
s, a, r, s'	Historical transition from the replay buffer
a'	Newly sampled from the current actor $\pi_\theta(\cdot \mid s')$
$\log \pi_\theta(a' \mid s')$	Evaluated using the current actor

Maintain a replay buffer:

$$\boxed{\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}.}$$

Gradient steps sample mini-batches from $\mathcal{D}$ rather than always requiring fresh rollouts. The same transitions can be reused repeatedly; multiple optimization steps may follow a single environment step.

Important limitation: off-policy does not mean that arbitrary distribution mismatch is harmless. If the buffer provides little coverage of important state-action regions, the neural critic can extrapolate

very poorly. Finite data, function approximation, and bootstrapping may jointly produce instability. SAC improves sample reuse but does not remove these challenges.

8. Step Seven: Deriving the Complete Critic Loss

8.1 From the soft Bellman target to TD regression

Sample a next action from the current policy, $a' \sim \pi_\theta(\cdot \mid s')$. A direct single-sample target would be:

$$y = r + \gamma \left[Q_\phi(s', a') - \alpha \log \pi_\theta(a' \mid s') \right].$$

The associated regression loss is:

$$J_Q(\phi) = \frac{1}{2} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s,a) - y \right)^2 \right].$$

This is not yet practical SAC: the target depends on the same network being updated, and the actor may exploit positive errors in the critic's estimates.

8.2 Target networks reduce target drift

Maintain a slowly changing target parameter vector $\bar{\phi}$ and a target critic $Q_{\bar{\phi}}$. Update them using Polyak averaging:

$$\boxed{\bar{\phi} \leftarrow (1-\tau)\bar{\phi} + \tau\phi, \qquad 0 < \tau \ll 1.} \tag{11}$$

A typical value is $\tau = 0.005$. The target network slows changes in the bootstrap target; it does not, by itself, eliminate value-estimation bias.

8.3 Clipped double Q mitigates overestimation

Suppose a learned value has an error $\hat{Q}(s,a) = Q(s,a) + \epsilon(s,a)$. An actor that prefers actions with high *estimated* Q may select actions mostly because of positive estimation error. For example, if two actions both truly have value 10 but are estimated at 12 and 8, the actor may favor the first one and amplify its accidental overestimate.

SAC therefore trains two independently parameterized critics:

$$Q_{\phi_1}(s,a), \qquad Q_{\phi_2}(s,a),$$

with corresponding target networks:

$$Q_{\bar{\phi}_1}(s,a), \qquad Q_{\bar{\phi}_2}(s,a).$$

A conservative minimum is used in the target:

$$\boxed{Q_{\text{target}}(s',a') = \min_{j=1,2} Q_{\bar{\phi}_j}(s',a').}$$

Taking the minimum reduces certain overestimation effects but can introduce underestimation; it does **not** make the estimator unbiased by construction.

8.4 Final critic target and losses

Let $d = 1$ denote a true terminal transition, not merely a time-limit truncation. The full target is:

$$\boxed{y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right], \quad a' \sim \pi_\theta(\cdot \mid s').} \tag{12}$$

Each critic fits the same target:

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\left(Q_{\phi_i}(s,a) - y\right)^2\right], \qquad i = 1, 2.} \tag{13}$$

$$\boxed{J_Q(\phi_1, \phi_2) = J_{Q_1}(\phi_1) + J_{Q_2}(\phi_2).} \tag{14}$$

Update $\phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} J_{Q_i}$. The target branch must have its gradients stopped: $y \leftarrow \mathrm{stopgrad}(y)$. For a **true terminal** transition, $d = 1$ and $y = r$. If an episode is truncated only because of an external time limit and the state is not terminal, bootstrapping should generally be retained.

9. Step Eight: Deriving the Actor Loss

The KL projection in Section 5 gave:

$$\pi_{\text{new}} = \arg\min_{\pi} \mathbb{E}_{a \sim \pi}\left[\alpha \log \pi(a \mid s) - Q(s,a)\right].$$

Replace the policy and value by a parameterized actor and critic, and draw states from the replay buffer:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s \sim \mathcal{D} \\ a \sim \pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right].} \tag{15}$$

A widely used twin-critic version is:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s \sim \mathcal{D} \\ a \sim \pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\phi_i}(s,a)\right].} \tag{16}$$

Some implementations instead use only the first critic for the actor update. Equation (16) contains two complementary objectives. Minimizing the expectation of $\alpha \log \pi_\theta$ encourages higher entropy, while minimizing $-Q_\phi$ favors high-value actions.

The key difference from PPO: PPO mainly uses sampled-action advantages and probability ratios. Continuous SAC aims to pass gradients through the action itself, using $\nabla_a Q_\phi(s,a)$ to communicate a local direction for improving action value. Because the action is randomly sampled, this requires a differentiable representation of the sampling procedure: the reparameterization trick.

10. Step Nine: The Reparameterization Trick

10.1 Turning parameter-dependent sampling into a differentiable function

Suppose the actor predicts a mean and standard deviation:

$$\mu_\theta(s), \qquad \sigma_\theta(s).$$

The unsquashed action is Gaussian:

$$u \sim \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2(s))\big).$$

Directly viewing sampling as $u \sim \pi_\theta$ makes the sampling distribution depend on θ. Instead, use the equivalent representation:

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{17}$$

Now the noise ϵ is independent of the parameters, and u_θ is differentiable with respect to θ. For example, $\partial u/\partial \mu = I$ and $\partial u/\partial \sigma = \mathrm{diag}(\epsilon)$.

10.2 A tanh-squashed Gaussian policy

Robot actions are generally bounded (for example, to $[-1, 1]^d$), whereas a Gaussian has unbounded support. SAC applies an elementwise hyperbolic tangent:

$$\boxed{a_\theta(s,\epsilon) = \tanh\big(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon\big).} \tag{18}$$

For a physical action interval $[a_{\min,i}, a_{\max,i}]$, one may further use $a_i^{\mathrm{env}} = c_i + b_i \tanh(u_i)$, where $c_i = (a_{\max,i} + a_{\min,i})/2$ and $b_i = (a_{\max,i} - a_{\min,i})/2$.

10.3 Rewriting the actor loss as a pathwise objective

Write $a = f_\theta(s, \epsilon)$ and replace policy-dependent action randomness with a fixed noise distribution:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ \epsilon\sim\mathcal{N}(0,I)}} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - Q_\phi(s, f_\theta(s,\epsilon))\right].} \tag{19}$$

Let $a_\theta = f_\theta(s,\epsilon)$. Differentiation gives:

$$\boxed{\nabla_\theta J_\pi = \mathbb{E}\left[\alpha \frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s, a_\theta) \frac{\partial a_\theta}{\partial \theta}\right].} \tag{20}$$

The derivative of $\log \pi_\theta(a_\theta \mid s)$ is a **total derivative**: the log-density depends on θ explicitly and through a_θ:

$$\frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) = \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} + \nabla_a \log \pi_\theta(a_\theta \mid s) \frac{\partial a_\theta}{\partial \theta}.$$

Thus the full gradient is:

$$\boxed{\begin{aligned}\nabla_\theta J_\pi = \mathbb{E}\Bigg[\alpha \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} \\ + \big(\alpha \nabla_a \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s, a_\theta)\big) \frac{\partial a_\theta}{\partial \theta}\Bigg].\end{aligned}} \tag{21}$$

The control-relevant term has a clear interpretation:

$$\underbrace{\nabla_a Q_\phi(s, a_\theta)}_{\text{direction of better actions}} \qquad \underbrace{\frac{\partial a_\theta}{\partial \theta}}_{\text{how to change actor parameters}}.$$

This gives SAC a direct connection to deterministic policy gradients such as DDPG: SAC exploits action derivatives of the critic but does so for a **reparameterizable stochastic policy**, together with entropy regularization. A pathwise gradient often has lower variance than a direct score-function estimator, although its usefulness still depends on critic quality.

11. Step Ten: Correcting the Tanh-Gaussian Log Probability

The Gaussian variable u and the transformed action $a = \tanh(u)$ do not have the same distribution. Therefore, $\log \mathcal{N}(a; \mu, \Sigma)$ is not the correct log-density for the squashed action. For an invertible change of variables $a = g(u)$:

$$\pi(a \mid s) = p_U(u \mid s) \left| \det \frac{\partial a}{\partial u} \right|^{-1},$$

$$\log \pi(a \mid s) = \log p_U(u \mid s) - \log \left| \det \frac{\partial a}{\partial u} \right|.$$

For elementwise $a_i = \tanh(u_i)$, $\partial a_i / \partial u_i = 1 - \tanh^2(u_i)$. The Jacobian is diagonal, and its determinant is $\prod_i (1 - \tanh^2(u_i))$. Hence:

$$\boxed{\log \pi_\theta(a \mid s) = \log \mathcal{N}(u; \mu_\theta(s), \Sigma_\theta(s)) - \sum_{i=1}^{d} \log \left(1 - \tanh^2(u_i)\right).} \tag{22}$$

If the environment action is additionally scaled as $a_i^{\text{env}} = c_i + b_i \tanh(u_i)$ with $b_i > 0$, then:

$$\boxed{\log \pi_\theta(a^{\text{env}} \mid s) = \log \mathcal{N}(u; \mu_\theta, \Sigma_\theta) - \sum_{i=1}^{d} \log \left(b_i (1 - \tanh^2(u_i))\right).} \tag{23}$$

Near saturation at ± 1, the direct $\log(1 - \tanh^2 u)$ expression can be numerically unstable. A stable identity is $\log(1 - \tanh^2 u) = 2(\log 2 - u - \operatorname{softplus}(-2u))$; implementations may alternatively add a small numerical stabilizer. Many implementations evaluate log-probabilities in normalized action coordinates and then scale actions for the environment. The crucial point is to use a consistent action-coordinate convention for density and entropy.

Motivation recap: both the SAC actor objective and automatic temperature tuning use $\log \pi$. Omitting the tanh Jacobian changes the objective and produces incorrect entropy gradients.

12. Step Eleven: Automatic Entropy Tuning

12.1 From fixed temperature to an entropy constraint

Manually choosing α is sensitive to reward scaling and action dimensionality. Rather than prescribing an entropy weight, we may require the policy to maintain a desired entropy level. A compact averaged formulation is:

$$\boxed{\begin{aligned} \max_{\pi} \quad & J_R(\pi) \\ \text{s.t.} \quad & \bar{\mathcal{H}}(\pi) \geq \mathcal{H}_{\text{target}}. \end{aligned}} \tag{24}$$

Here $\bar{\mathcal{H}}$ is the average policy entropy under the relevant state visitation distribution. The original automatic-temperature derivation uses per-time-step expected entropy constraints; we use the average form to convey the same design principle without unnecessary notation.

Introduce a nonnegative Lagrange multiplier α:

$$\boxed{\mathcal{L}(\pi, \alpha) = J_R(\pi) + \alpha\big(\bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}\big).} \tag{25}$$

This yields the optimization form:

$$\boxed{\max_{\pi} \min_{\alpha \geq 0} \mathcal{L}(\pi, \alpha).} \tag{26}$$

For fixed α, optimizing the policy recovers the maximum-entropy objective. For fixed policy, optimizing α yields the dual update that aims to enforce the entropy target.

12.2 The temperature loss and its direction of change

Since $\mathcal{H}(\pi) = -\mathbb{E} \log \pi$, the dual-temperature loss is:

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta} \left[\alpha \left(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}} \right) \right].} \tag{27}$$

Equivalently, $J_\alpha = -\mathbb{E}[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})]$. Differentiating:

$$\boxed{\frac{\partial J_\alpha}{\partial \alpha} = \bar{\mathcal{H}}(\pi_\theta) - \mathcal{H}_{\text{target}}.} \tag{28}$$

Thus:

$$\boxed{\begin{cases} \bar{\mathcal{H}}(\pi) < \mathcal{H}_{\text{target}} & \Rightarrow \alpha \uparrow, \\ \bar{\mathcal{H}}(\pi) > \mathcal{H}_{\text{target}} & \Rightarrow \alpha \downarrow. \end{cases}} \tag{29}$$

If policy entropy falls below the target, gradient descent increases α and encourages exploration. If entropy exceeds the target, α decreases and lets the actor concentrate more strongly on high-Q actions.

12.3 Why optimize log-alpha?

Directly updating α may produce negative values, which are inadmissible. Set $\eta = \log \alpha$ and $\alpha = e^\eta > 0$. Substituting exactly into the dual loss gives:

$$J_\eta(\eta) = -\mathbb{E} \left[e^\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}} \right) \right].$$

Its gradient includes the factor $e^\eta = \alpha$. Many popular implementations instead use a surrogate with a different gradient scale but the same interior stationary condition:

$$\boxed{J_\eta^{\text{surrogate}}(\eta) = -\mathbb{E} \left[\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}} \right) \right].} \tag{30}$$

During temperature optimization, detach the log-probability tensor so that the temperature loss does not unintentionally update actor parameters.

12.4 Choosing target entropy

For a normalized d-dimensional continuous action, a popular heuristic is $\mathcal{H}_{\text{target}} = -d$. For $d = 12$, one might initially try -12. This is not a theorem or universally optimal choice. Differential entropy can be negative and depends on the coordinate system used for action density. High-dimensional hands, robot arms, and action-chunk policies may require a task-specific target-entropy search.

13. Step Twelve: Putting Together the Complete Modern SAC Algorithm

13.1 Components and three optimization problems

A modern continuous-action SAC implementation typically maintains:

Component	Parameters or object	Purpose
Actor	θ	Generates reparameterizable stochastic actions
Critics 1 and 2	ϕ_1, ϕ_2	Estimate soft Q and mitigate overestimation
Target critics	$\bar{\phi}_1, \bar{\phi}_2$	Produce slowly changing TD targets
Temperature	α or $\eta = \log \alpha$	Regulates entropy weight
Replay buffer	$\mathcal{D}$	Stores historical (s, a, r, s', d) transitions

The three optimization problems are:

$$\boxed{\min_{\phi_1,\phi_2} J_Q, \qquad \min_{\theta} J_\pi, \qquad \min_{\alpha \geq 0} J_\alpha.}$$

13.2 Complete sequence for one update

Step 1: interact with the environment. At state s_t, sample $a_t \sim \pi_\theta(\cdot \mid s_t)$; execute the action and store $(s_t, a_t, r_t, s_{t+1}, d_t)$. During early training, random actions can be used to obtain initial coverage.

Step 2: sample past transitions. Draw a mini-batch $B = \{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^N$ from $\mathcal{D}$.

Step 3: compute targets. At the next states, sample $a'_i \sim \pi_\theta(\cdot \mid s'_i)$ using the current actor and compute:

$$\boxed{y_i = r_i + \gamma(1 - d_i)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s'_i, a'_i) - \alpha \log \pi_\theta(a'_i \mid s'_i)\right].}$$

Step 4: update both critics. Each critic fits the same target:

$$J_{Q_j} = \frac{1}{2N} \sum_{i=1}^{N} (Q_{\phi_j}(s_i, a_i) - y_i)^2.$$

Step 5: update the actor. Resample $\epsilon_i \sim \mathcal{N}(0, I)$ and set $\tilde{a}_i = f_\theta(s_i, \epsilon_i)$. Minimize:

$$\boxed{J_\pi = \frac{1}{N}\sum_{i=1}^{N}\left[\alpha \log \pi_\theta(\tilde{a}_i \mid s_i) - \min_j Q_{\phi_j}(s_i, \tilde{a}_i)\right].}$$

The critic **weights** should be frozen during the actor step, but gradients through the critic **input action** must be retained: $\nabla_a Q_\phi$ is necessary.

Step 6: update temperature. Adjust α using the difference between $-\log \pi_\theta$ and the target entropy. Detach the log-probability during the temperature update.

Step 7: update target critics. For each critic, apply $\bar{\phi}_j \leftarrow (1-\tau)\bar{\phi}_j + \tau\phi_j$. Repeat the cycle.

13.3 Instructional pseudocode

```
initialize actor pi_theta
initialize critics Q1, Q2
initialize target critics Q1_targ, Q2_targ from Q1, Q2
initialize replay buffer D and log_alpha

for each environment interaction:
    action = pi_theta.sample(obs)  # warm-up may use random actions
    next_obs, reward, terminated, truncated = env.step(action)
    D.add(obs, action, reward, next_obs, terminated)
    obs = env.reset() if (terminated or truncated) else next_obs

    for each gradient update:
        bs, ba, br, bs_next, bd = D.sample(batch_size)
        alpha = exp(log_alpha)

        # 1) Critic target: no gradient through target branch
        with no_grad():
            a_next, logp_next = pi_theta.rsample(bs_next)
            q_next = min(Q1_targ(bs_next, a_next),
                         Q2_targ(bs_next, a_next))
            y = br + gamma * (1 - bd) * (q_next - alpha * logp_next)

        loss_q1 = mean((Q1(bs, ba) - y) ** 2)
        loss_q2 = mean((Q2(bs, ba) - y) ** 2)
        update(Q1, loss_q1)
        update(Q2, loss_q2)

        # 2) Actor: freeze Q weights, keep dQ/da
        a_pi, logp_pi = pi_theta.rsample(bs)
        q_pi = min(Q1(bs, a_pi), Q2(bs, a_pi))
        loss_pi = mean(alpha.detach() * logp_pi - q_pi)
        update(pi_theta, loss_pi)

        # 3) Automatic entropy tuning (log-alpha surrogate)
        loss_alpha = -mean(log_alpha *
                           (logp_pi.detach() + target_entropy))
        update(log_alpha, loss_alpha)

        # 4) Polyak target update
        for Q, Q_targ in [(Q1, Q1_targ), (Q2, Q2_targ)]:
            Q_targ.params = (1 - tau) * Q_targ.params + tau * Q.params
```

> **Implementation warning:** this is explanatory pseudocode, not a complete runnable library. Production code must correctly handle optimizer gradient resets, parameter freezing, action

> scaling, density corrections, exploration warm-up, environment resets, and batch dimensions. It must also distinguish `terminated` from `truncated`.

14. Original SAC v1 Versus Modern SAC

14.1 Original SAC v1: explicitly learning a V-network

The original SAC formulation trained an additional state-value network $V_\psi(s)$, besides the actor and two Q-functions:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].$$

One can therefore fit:

$$\boxed{J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \left(\min_i Q_{\phi_i}(s,\tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s)\right)\right)^2\right].}$$

When training V_ψ, the target expression is treated as a constant. Its Q-function backup is:

$$\boxed{y_Q = r + \gamma(1-d)V_{\bar{\psi}}(s').}$$

14.2 Modern SAC: remove the explicit V-network

The identity:

$$V^\pi(s') = \mathbb{E}_{a'\sim\pi}\big[Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\big]$$

can be approximated directly with a newly sampled action:

$$V^\pi(s') \approx \min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s').$$

Substitution gives the target in Equation (12) without requiring an additional V-network. This removes one set of function-approximation errors and simplifies the model. Today, the phrase "standard SAC" usually refers to this modern twin-Q version, frequently combined with automatic temperature adjustment.

15. Fundamental Differences Between SAC and PPO

15.1 Comparing policy-update equations

PPO uses sampled-action advantage estimates and old-to-new policy likelihood ratios:

$$\boxed{J_{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right],}$$

where $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$. The optimizer **maximizes** this surrogate objective, and clipping restrains large policy changes. Its typical policy-gradient signal is related to $\nabla_\theta \log \pi_\theta(a \mid s)\, A^\pi(s,a)$.

SAC instead **minimizes**:

$$\boxed{J_{\mathrm{SAC}}(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a_\theta \mid s) - Q_\phi(s, a_\theta)\right], \quad a_\theta = f_\theta(s, \epsilon).}$$

Its gradient includes $\nabla_a Q_\phi(s, a_\theta)\, \partial a_\theta / \partial \theta$. PPO generally does not require $\nabla_a Q$; standard continuous-action SAC does.

15.2 Comparison table

Dimension	PPO	SAC
Data regime	On-policy	Off-policy
Actor	Usually stochastic	Reparameterizable stochastic policy
Critic	Commonly $V(s)$	Commonly $Q(s, a)$
Policy signal	Advantage and probability ratio	Q-value, action gradient, entropy
Main gradient estimator	Score-function estimator	Pathwise / reparameterization
Reuse of old data	Limited; fresh rollouts dominate	Replay buffer with repeated sampling
Entropy term	Common but optional	Central to the objective
Stabilization	Clipped policy update	Double Q and target critics
Sample efficiency	Often lower	Often higher
Typical strength	High-throughput parallel simulation	Expensive environment interactions
Main challenge	Needs many new transitions	Critic extrapolation and bootstrapping stability

PPO clipping and SAC double Q are **different stabilization mechanisms**: the former primarily restricts policy probability changes, while the latter addresses Q overestimation. Standard SAC does not impose the same explicit probability-ratio clipping as PPO.

15.3 Why PPO remains popular in robot simulation

On platforms such as Isaac Lab, thousands of parallel simulated environments may generate substantial experience at low marginal cost. PPO can exploit high-throughput rollouts and achieve excellent training speed. SAC reuses samples, but frequent critic updates, bootstrapped learning, and replay-buffer access also incur computational costs. Hence:

$$\boxed{\text{Sample Efficiency} \neq \text{Wall-Clock Efficiency}.}$$

Contact nonlinearity, sparse rewards, and major state-distribution shifts can also make SAC critics difficult to train. Conversely, when every physical robot interaction is expensive, SAC's off-policy reuse is often attractive, subject to safe exploration and coverage constraints.

16. Final Summary: Five Essential Formula Groups

(1) Maximum-entropy objective:

$$\boxed{J(\pi) = \mathbb{E}_{\tau}\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big)\right].}$$

Motivation: maximize reward while retaining exploration through entropy regularization.

(2) Soft Bellman equation:

$$\boxed{Q^{\pi}(s,a) = \mathbb{E}\left[r + \gamma\big(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s')\big)\right].}$$

Motivation: recursively decompose the maximum-entropy objective for TD learning.

(3) KL-based soft policy improvement:

$$\boxed{\pi_{\text{new}} = \arg\min_{\pi} D_{\text{KL}}\left(\pi(\cdot \mid s) \,\middle\|\, \frac{\exp(Q^{\pi_{\text{old}}}(s,\cdot)/\alpha)}{Z(s)}\right),}$$

which is equivalent to minimizing $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q^{\pi_{\text{old}}}(s,a)]$. *Motivation:* approach a high-value Boltzmann distribution without evaluating its normalizing constant.

(4) Modern SAC critic and actor:

$$\boxed{J_{Q_i} = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\big[(Q_{\phi_i}(s,a) - y)^2\big], \quad y = r + \gamma(1-d)\big[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_{\theta}(a' \mid s')\big],}$$

$$\boxed{J_{\pi} = \mathbb{E}_{s,\epsilon}\big[\alpha \log \pi_{\theta}(f_{\theta}(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_{\theta}(s,\epsilon))\big].}$$

Motivation: make soft policy iteration practical with trainable networks, historical transitions, twin critics, and pathwise gradients.

(5) Automatic entropy tuning:

$$\boxed{J_{\alpha} = -\mathbb{E}\big[\alpha(\log \pi_{\theta}(a \mid s) + \mathcal{H}_{\text{target}})\big].}$$

Motivation: automatically adapt exploration pressure to a desired entropy level.

The single most important PPO/SAC distinction

PPO mainly uses $\nabla_{\theta} \log \pi_{\theta}(a \mid s) A^{\pi}(s,a)$ to change the **probabilities of sampled actions**. Continuous SAC uses $\nabla_a Q_{\phi}(s,a)\, \partial a_{\theta}/\partial \theta$, together with the entropy gradient, to change the **differentiable action generator** toward higher-value actions.

The main logical progression is:

$$\boxed{\begin{array}{c}\text{Maximum-Entropy RL} \longrightarrow \text{Soft Bellman} \longrightarrow \text{KL Policy Improvement} \\ \longrightarrow \text{Reparameterization} \longrightarrow \text{Off-Policy Actor-Critic}\end{array}}$$

Double Q, target critics, and automatic temperature tuning make this theory useful for challenging practical continuous-control problems.

References

1. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** ICML. https://arxiv.org/abs/1801.01290 — Theory of soft policy iteration, KL improvement, and stochastic actors.
2. Haarnoja, T., Zhou, A., Hartikainen, K., et al. (2018). **Soft Actor-Critic Algorithms and Applications.** https://arxiv.org/abs/1812.05905 — Automatic temperature adjustment and the modern SAC design.
3. OpenAI Spinning Up. **Soft Actor-Critic.** https://spinningup.openai.com/en/latest/algorithms/sac.html — Modern SAC losses and implementation details.

Document note: This Markdown is the single source used to generate the corresponding English PDF. Section structure, equations, tables, and references are identical across the two formats.